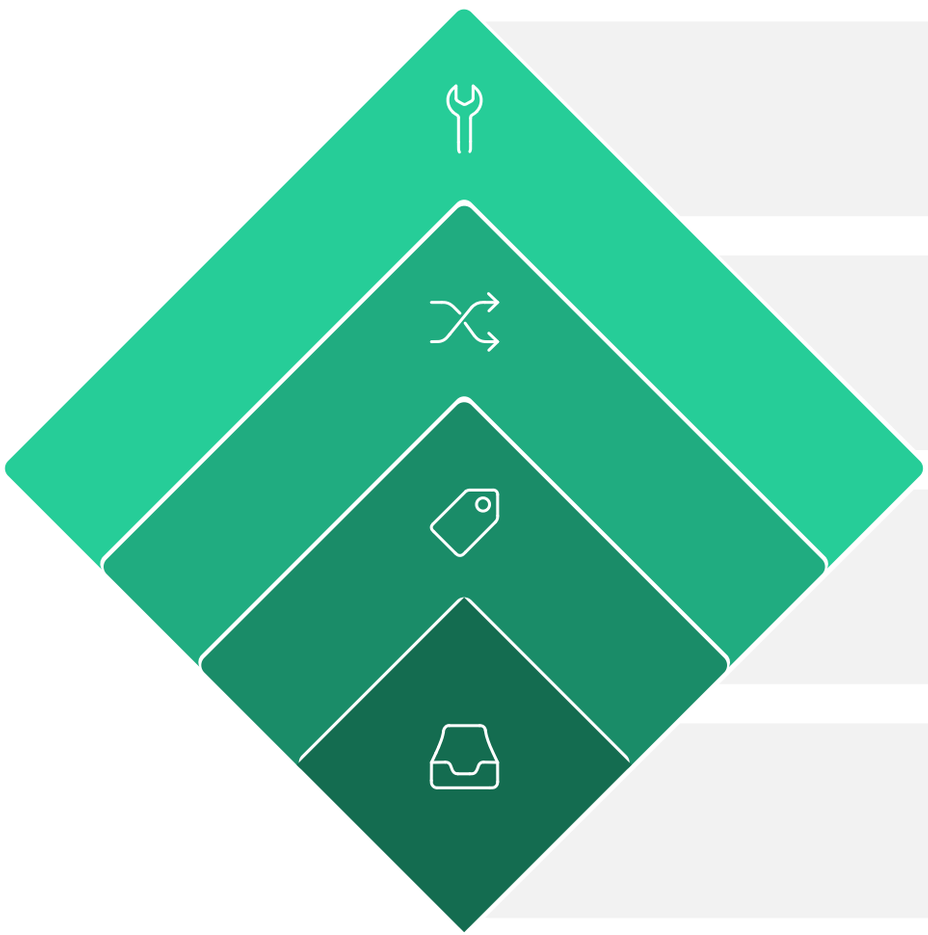
AI Support Engineer Copilot

LangGraph + LangSmith Support Escalation Workflow

Automating support triage, troubleshooting, and escalation workflows.



Troubleshooting

Investigate across branches

Conditional Routing

Route to specialized workflows

Classification

Assign category and priority

Support Issue

Issue intake and details

AI Support Engineer Copilot

Using LangGraph and LangSmith to automate support issue triage and escalation preparation.

Classify Issues

Automatic categorization of incoming support tickets

Recommend Steps

Targeted troubleshooting per issue type

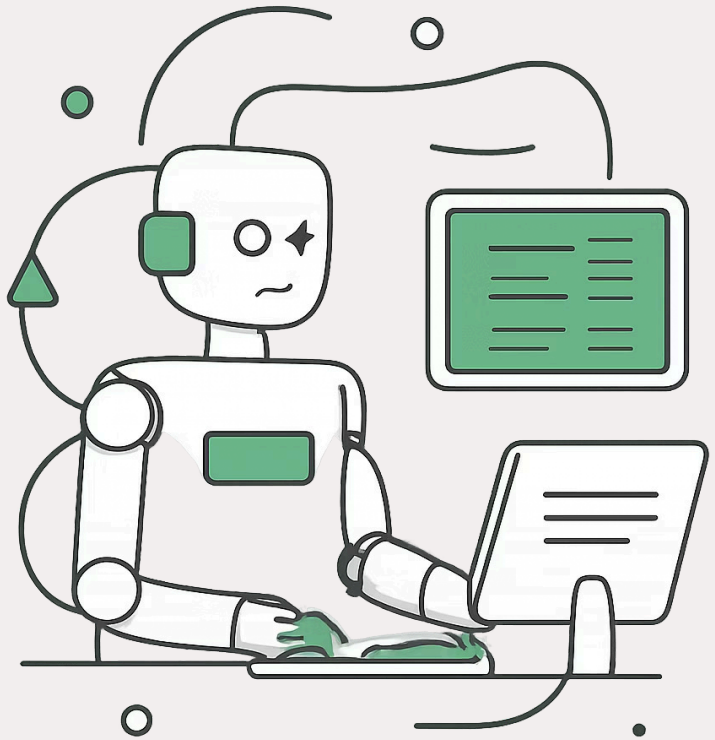
Generate Summaries

Escalation-ready documentation

Measure Quality

LangSmith evaluation experiments

LANGGRAPH | LANGSMITH | GPT-4O-MINI | PYTHON



Assignment Requirements & Results

LangGraph Workflow

Classification, routing, troubleshooting, and summary generation nodes

Dataset Creation

25 support issues across 5 categories

LangSmith Evaluations

Measured workflow performance end-to-end

UI + SDK

Datasets and traces in UI; evaluation experiments via SDK

96%

Classification Accuracy

100%

Summary Quality Score

0%

Execution Errors

The Problem

Support engineers perform the same initial steps on every ticket — manually, every time.

1

Read the Issue

Understand the incoming problem

2

Determine Type

Classify the issue category

3

Follow Steps

Apply troubleshooting procedures

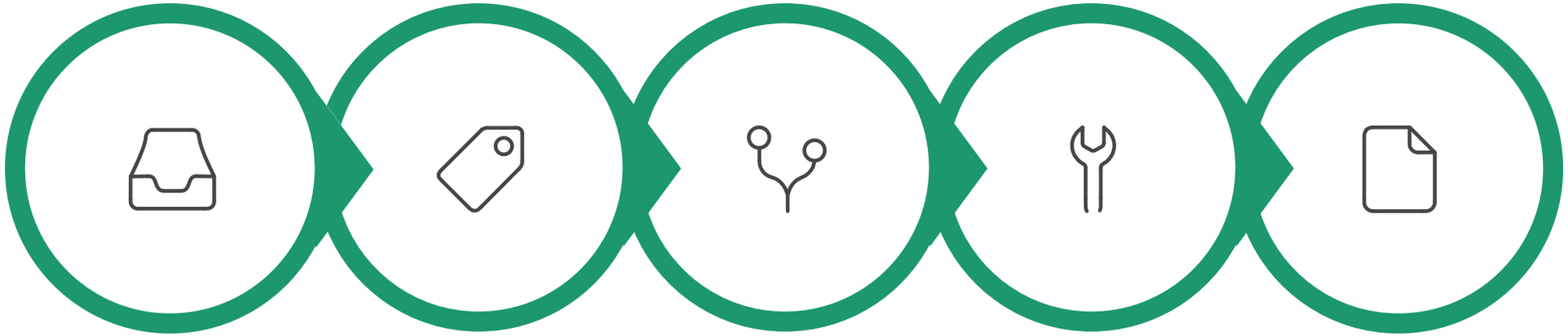
4

Write Summary

Draft escalation documentation

ⓘ Goal: Build a workflow that helps support engineers complete these steps **more consistently and more quickly**.

Solution Architecture



Conditional routing replaced a single monolithic troubleshooting node — different issue types require different approaches, and this unlocks one of LangGraph's core capabilities.

v1 Design: Single troubleshooting node for all issue types

v2 Design: Conditional routing to specialized troubleshooters per category

LangGraph Workflow — Implementation

Shared State

- issue
- category
- troubleshooting
- escalation_summary

Key Concepts

- **State** — stores shared information across nodes
- **Nodes** — perform discrete units of work
- **Edges** — connect nodes in sequence
- **Conditional Routing** — chooses the next path dynamically

Node Inventory

01

classify_issue()

02

route_issue()

03

authentication_troubleshooter()

04

ssl_troubleshooter()

05

kubernetes_troubleshooter()

06

connectivity_troubleshooter()

07

performance_troubleshooter()

08

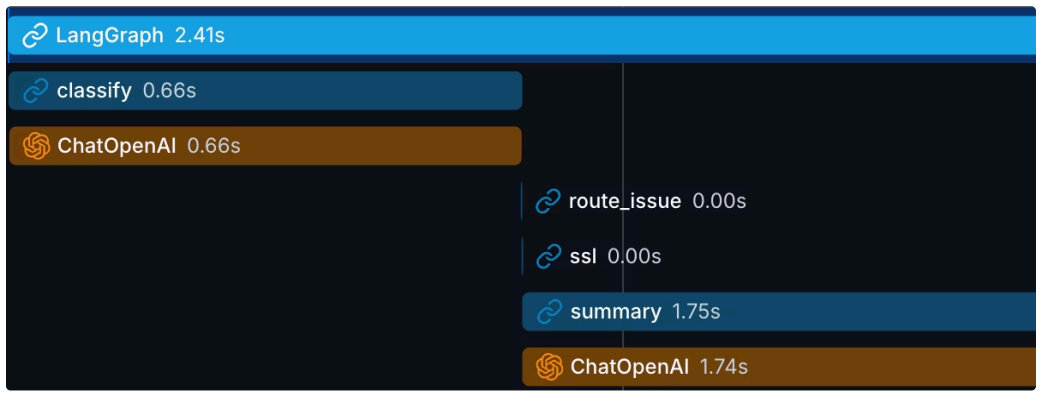
generate_summary()

Using LangSmith

LangSmith provided full observability into every step of the workflow from input to output.

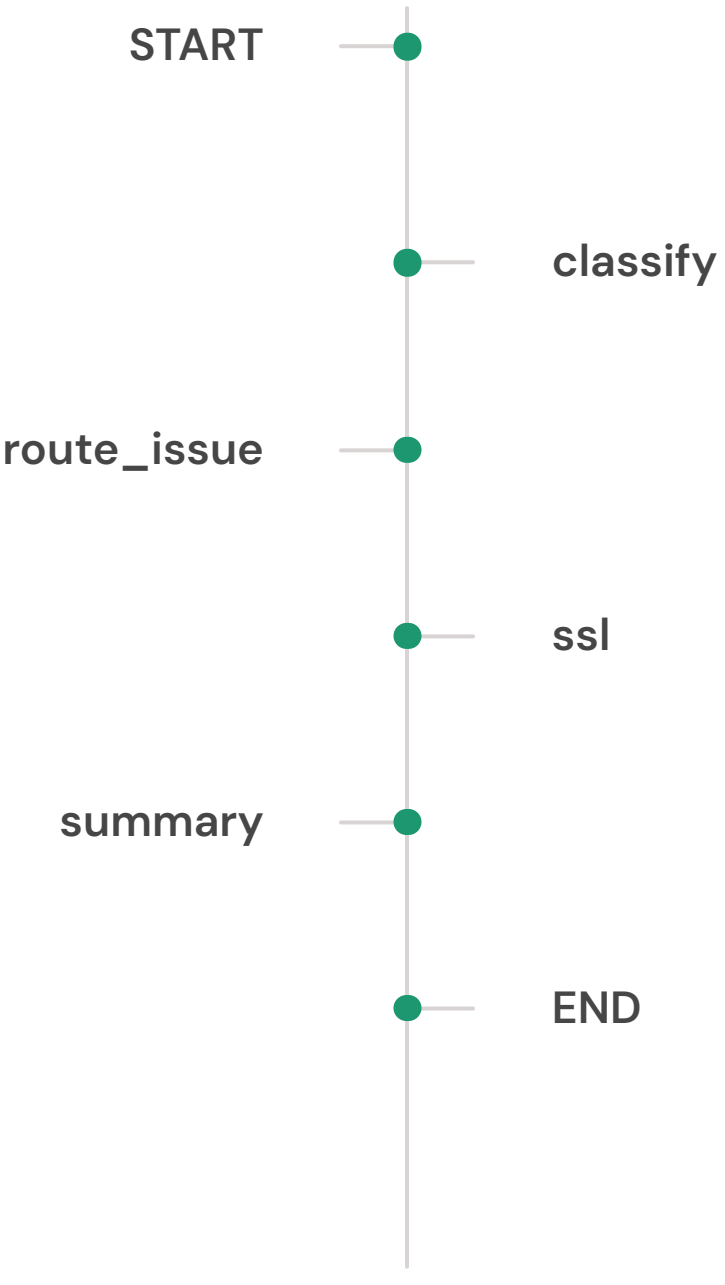
How I Used LangSmith

- Followed each issue through the workflow
- Verified classification and routing decisions
- Inspected inputs and outputs at every node
- Measured execution latency
- Debugged workflow behavior using traces



This made it easy to verify that issues were reaching the correct troubleshooting path.

Example Trace Outcome



Dataset & Evaluation Approach

Dataset

25 Issues

Total support tickets

5 Categories

5 examples each

Categories

- Authentication
- SSL
- Kubernetes
- Connectivity
- Performance

Evaluation #1 — Deterministic

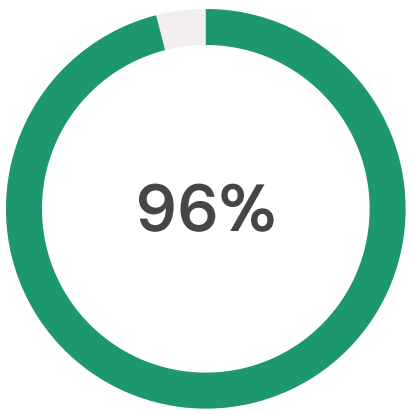
Compared **Predicted Category** vs **Expected Category** using code-based exact match scoring.

Evaluation #2 — LLM-as-Judge

Checked whether the escalation summary:

- Clearly explained the problem
- Included troubleshooting actions
- Included a recommended next step
- Was relevant to the issue

Results



Classification Accuracy

24 of 25 issues correctly classified

Summary Quality

All summaries passed LLM-as-Judge evaluation

⚠ **Why the Misclassification Happened**
Issue: *"Node memory pressure is affecting workloads."*
Expected: **Kubernetes** Predicted: **Performance**

Some support issues naturally overlap multiple categories. Evaluations help identify these edge cases

Error Rate: 0%
Zero execution errors across all 25 evaluation runs. The workflow completed successfully on every issue.

Key Learnings & Friction Points

Things That Took Time

State Flow

Understanding how state moves between nodes

Local vs LangSmith

Difference between local testing and LangSmith datasets

Evaluation Types

Deterministic vs LLM-as-Judge evaluation strategies

UI + SDK

How the LangSmith UI and SDK work together

What Helped

- Building the workflow **one step at a time**
- Using LangSmith traces to **follow execution**
- Starting simple before adding **advanced evaluation**

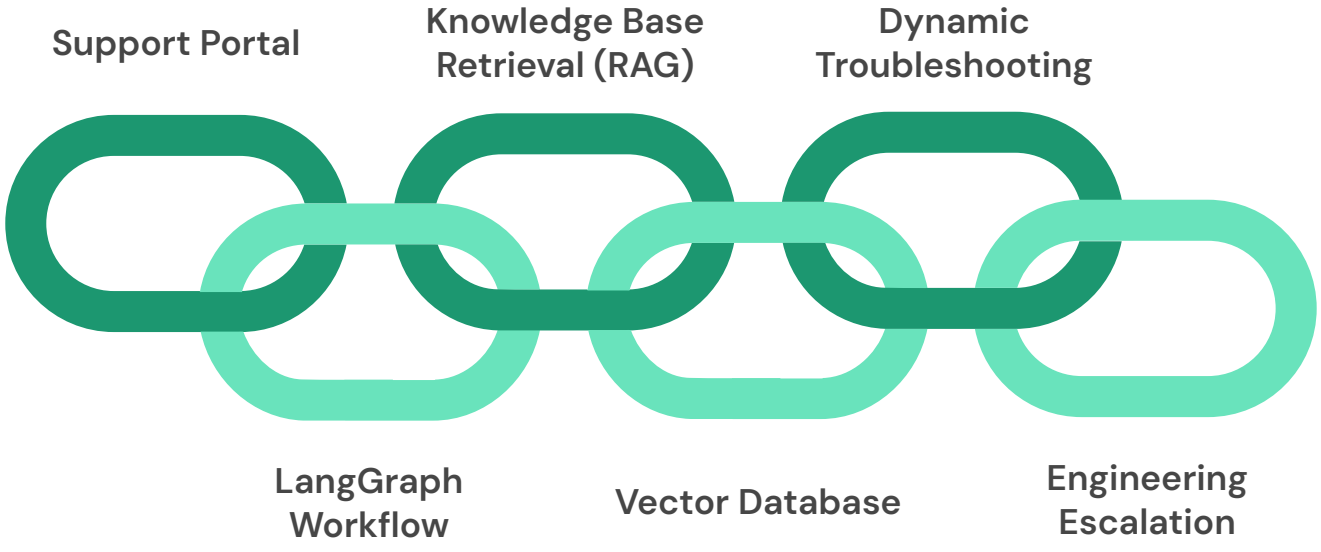
Biggest Takeaway: Evaluating AI outputs is just as important as generating them.

Future Improvements & Production Vision

Short-Term

- Expand the dataset beyond 25 issues
- Improve classification prompts for edge cases
- Add more support categories

Production Architecture



Final Takeaway: This project demonstrated how LangGraph and LangSmith work together to build, observe, and evaluate AI workflows — and how evaluation surfaces weaknesses that improve system quality over time.